2026

PETCARE

Application de gestion et de suivi des animaux domestiques

Tristan DZIOCH

III. Modélisation de la Base de Données

3.1 Introduction à la méthode MERISE

Dans le cadre de ce projet, la modélisation de la base de données est réalisée selon la méthode MERISE. Cette approche consiste à concevoir la base en trois étapes complémentaires : le Modèle Conceptuel de Données (MCD), le Modèle Logique de Données (MLD) puis le Modèle Physique de Données (MPD). Cette progression du conceptuel vers le physique permet de garantir que la base répond aux besoins fonctionnels, tout en assurant la cohérence des relations, la normalisation des données et l'adaptation au SGBD retenu.

3.2 Dictionnaire des données

Dans le cadre de la conception de la base de données, j'ai élaboré un dictionnaire des données afin de recenser, structurer et définir l'ensemble des informations manipulées par l'application. Ce dictionnaire constitue un outil essentiel permettant d'assurer la cohérence, la compréhension et la maintenabilité du modèle de données.

Il regroupe, pour chaque entité, les attributs associés ainsi que leur type, leur signification et leurs contraintes éventuelles (clé primaire, clé étrangère, unicité). Cette étape permet notamment d'éviter les ambiguïtés, de garantir une bonne normalisation des données et de faciliter la transition vers les modèles logique et physique.

La base de données du projet repose sur cinq entités principales : **Utilisateur**, **Animal**, **Evenement**, **Type_Evenement** et **Partage_Animal**.

Entité : Utilisateur

Attribut	Type	Description	Contraintes
id_utilisateur	INT	Identifiant unique de l'utilisateur	PK, AUTO_INCREMENT
nom	VARCHAR(255)	Nom de l'utilisateur	NOT NULL
prenom	VARCHAR(255)	Prénom de l'utilisateur	NOT NULL
email	VARCHAR(255)	Adresse email utilisée pour la connexion	NOT NULL, UNIQUE
mot_de_pass_hash	VARCHAR(255)	Mot de passe chiffré de l'utilisateur	NOT NULL
date_inscription	DATETIME	Date de création du compte	NOT NULL
role	VARCHAR(50)	Rôle de l'utilisateur (admin, utilisateur...)	NOT NULL
Photo_url	VARCHAR(255)	URL de la photo de l'user	NULL

Entité : Animal

Attribut	Type	Description	Contraintes
id_animal	INT	Identifiant unique de l'animal	PK, AUTO_INCREMENT
nom	VARCHAR(255)	Nom de l'animal	NOT NULL
espece	VARCHAR(255)	Espèce de l'animal (chien, chat...)	NOT NULL
race	VARCHAR(100)	Race de l'animal	NULL
date_naissance	DATE	Date de naissance de l'animal	NULL
sexe	ENUM('M','F','Inconnu')	Sexe de l'animal	NULL
photo_url	VARCHAR(255)	URL de la photo de l'animal	NULL
id_proprietaire	INT	Référence au propriétaire de l'animal	FK → Utilisateur(id_utilisateur), NOT NULL

Entité : Evenement

Attribut	Type	Description	Contraintes
id_evenement	INT	Identifiant unique de l'événement	PK, AUTO_INCREMENT
id_animal	INT	Animal concerné par l'événement	FK → Animal(id_animal), NOT NULL
id_type_evenement	INT	Type de l'événement	FK → Type_Evenement(id_type_evenement), NOT NULL
id_createur	INT	Utilisateur ayant créé l'événement	FK → Utilisateur(id_utilisateur), NOT NULL
date_heure_evenement	DATETIME	Date et heure de l'événement	NOT NULL
statut	VARCHAR(50)	Statut de l'événement (prévu, effectué...)	NOT NULL
rappel_jours_avant	INT	Nombre de jours avant rappel	NULL
rappel_actif	BOOLEAN	Indique si le rappel est actif	NOT NULL
commentaire	VARCHAR(500)	Commentaire associé à l'événement	NULL
date_creation	DATETIME	Date de création de l'événement	NOT NULL
date_modification	DATETIME	Date de dernière modification	NULL

Entité : Type_Evenement

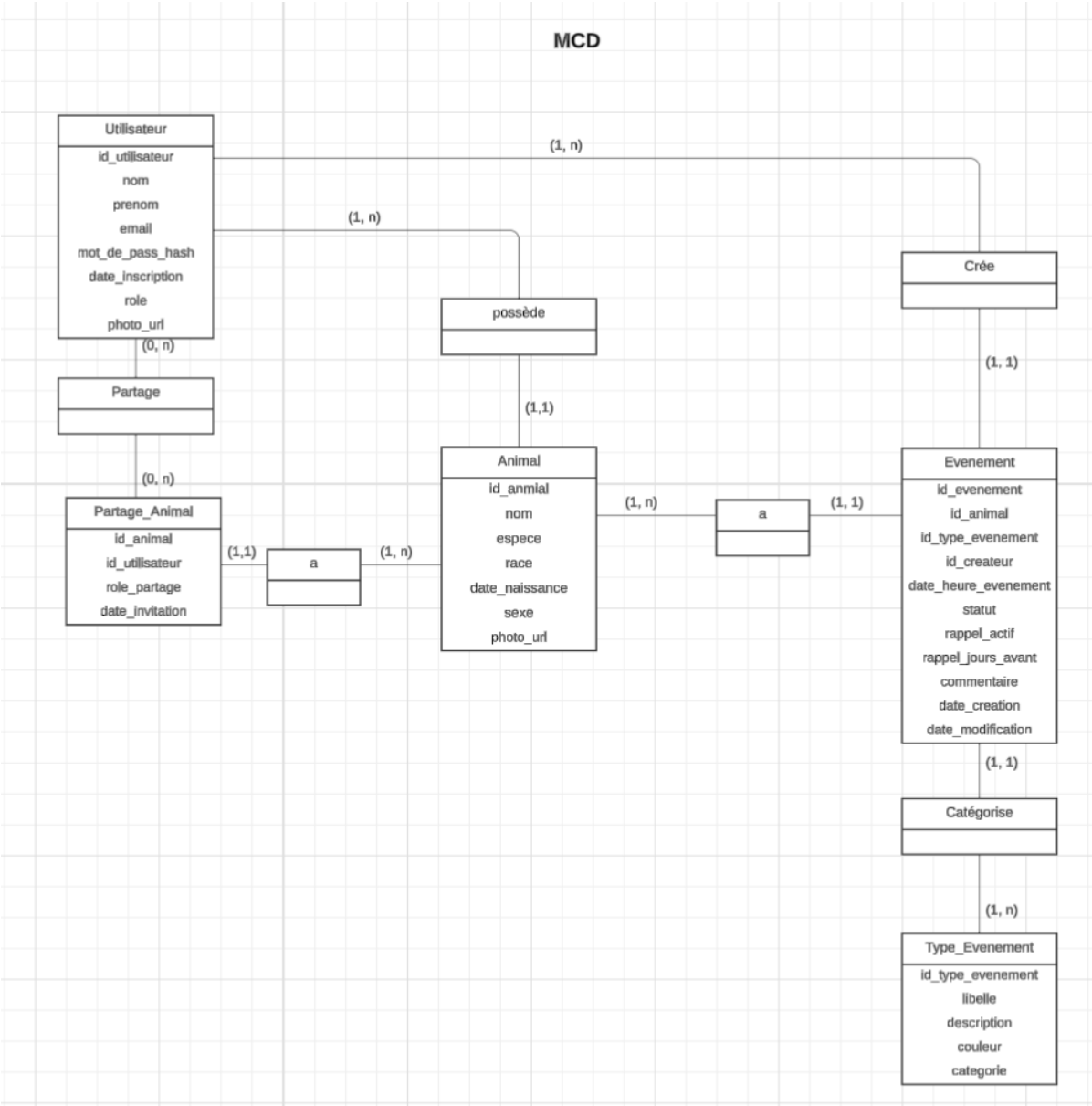
Attribut	Type	Description	Contraintes
id_type_evenement	INT	Identifiant unique du type d'événement	PK, AUTO_INCREMENT
libelle	VARCHAR(255)	Nom du type d'événement	NOT NULL
description	VARCHAR(255)	Description du type d'événement	NULL
couleur	VARCHAR(255)	Couleur associée (affichage)	NULL
categorie	VARCHAR(255)	Catégorie du type d'événement	NULL

Entité : Partage_Animal

Attribut	Type	Description	Contraintes
id_animal	INT	Animal partagé	PK, FK → Animal(id_animal)
id_utilisateur	INT	Utilisateur ayant accès à l'animal	PK, FK → Utilisateur(id_utilisateur)
role_partage	VARCHAR(50)	Rôle attribué (lecture, modification...)	NOT NULL
date_invitation	DATE	Date de partage de l'animal	NOT NULL

Pour conclure, le dictionnaire des données permet de structurer précisément l'ensemble des informations du système et constitue une étape fondamentale dans la conception de la base de données. Il garantit la cohérence du modèle, facilite sa compréhension et sert de référence pour la création du modèle logique et du modèle physique.

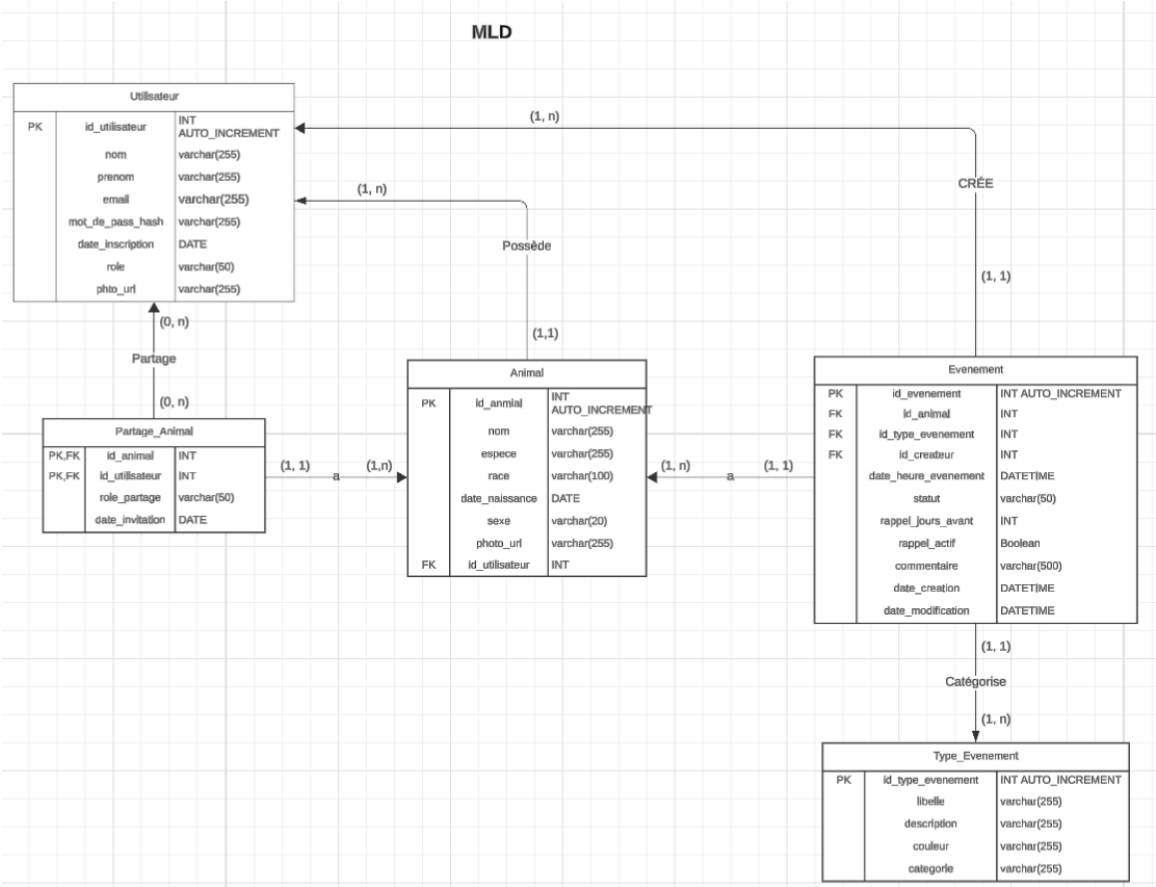
a. Modèle Conceptuel de Données (MCD)



Le MCD met en évidence les relations suivantes. Un utilisateur propriétaire peut posséder plusieurs animaux, tandis qu'un animal appartient à un seul propriétaire. Chaque animal peut avoir plusieurs événements, et chaque événement est rattaché à un seul animal. Un événement est catégorisé par un type d'événement unique, et un même type peut être réutilisé par plusieurs événements. Enfin, chaque événement possède un créateur unique (utilisateur), garantissant la traçabilité des actions.

La collaboration est assurée par l'association plusieurs-à-plusieurs entre Utilisateur et Animal via l'entité Partage_Animal, permettant de gérer un accès restreint ou étendu selon un rôle de partage.

b. **Modèle Logique de Données (MLD)**



La traduction du MCD en MLD aboutit à la création de cinq tables. Utilisateur et Animal sont liées par une clé étrangère id_proprietaire dans la table Animal. La table Evenement contient les clés étrangères vers Animal, Type_Evenement et Utilisateur (créateur), ce qui respecte les cardinalités du MCD. La table Partage_Animal est une table de liaison utilisant une clé primaire composite (id_animal, id_utilisateur), ce qui garantit l'unicité d'un partage entre un utilisateur et un animal.

c. Modèle Physique de Données (MPD)

```
-- ==========================
-- TABLE : utilisateur
-- ==========================

CREATE TABLE utilisateur (
 id_utilisateur INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 nom VARCHAR(255) NOT NULL,
 prenom VARCHAR(255) NOT NULL,
 email VARCHAR(255) NOT NULL UNIQUE,
 mot_de_passe_hash VARCHAR(255) NOT NULL,
 date_inscription DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
 role VARCHAR(50) NOT NULL DEFAULT 'utilisateur'
) ENGINE=InnoDB;




-- ==========================
-- TABLE : animal
-- ==========================

CREATE TABLE animal (
 id_animal INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 nom VARCHAR(255) NOT NULL,
 espece VARCHAR(255) NOT NULL,
 race VARCHAR(100) NULL,
 date_naissance DATE NULL,
 sexe ENUM('M','F','Inconnu') NOT NULL DEFAULT 'Inconnu',
 photo_url VARCHAR(255) NULL,
 id_proprietaire INT UNSIGNED NOT NULL,

 CONSTRAINT fk_animal_proprietaire
 FOREIGN KEY (id_proprietaire)
 REFERENCES utilisateur(id_utilisateur)
 ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE INDEX idx_animal_proprietaire
ON animal(id_proprietaire);



-- ==========================
-- TABLE : type_evenement
```

```sql
-- ==========================

CREATE TABLE type_evenement (
 id_type_evenement INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 libelle VARCHAR(255) NOT NULL,
 description TEXT NULL,
 couleur VARCHAR(50) NULL,
 categorie VARCHAR(100) NULL
) ENGINE=InnoDB;

-- ==========================
-- TABLE : evenement
-- ==========================

CREATE TABLE evenement (
 id_evenement INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 id_animal INT UNSIGNED NOT NULL,
 id_type_evenement INT UNSIGNED NOT NULL,
 id_createur INT UNSIGNED NOT NULL,
 date_heure_evenement DATETIME NOT NULL,
 statut VARCHAR(50) NOT NULL DEFAULT 'a_confirmer',
 rappel_jours_avant INT NOT NULL DEFAULT 0,
 rappel_actif BOOLEAN NOT NULL DEFAULT FALSE,
 commentaire TEXT NULL,
 date_creation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
 date_modification DATETIME NULL,

 CONSTRAINT fk_evenement_animal
 FOREIGN KEY (id_animal)
 REFERENCES animal(id_animal)
 ON DELETE CASCADE,

 CONSTRAINT fk_evenement_type
 FOREIGN KEY (id_type_evenement)
 REFERENCES type_evenement(id_type_evenement),

 CONSTRAINT fk_evenement_createur
 FOREIGN KEY (id_createur)
 REFERENCES utilisateur(id_utilisateur)
) ENGINE=InnoDB;

CREATE INDEX idx_evenement_animal_date
ON evenement(id_animal, date_heure_evenement);

CREATE INDEX idx_evenement_createur
ON evenement(id_createur);
```

```
CREATE INDEX idx_evenement_type
ON evenement(id_type_evenement);

-- ========================
-- TABLE : partage_animal
-- ========================

CREATE TABLE partage_animal (
 id_animal INT UNSIGNED NOT NULL,
 id_utilisateur INT UNSIGNED NOT NULL,
 role_partage VARCHAR(50) NOT NULL DEFAULT 'consultation',
 date_invitation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

 PRIMARY KEY (id_animal, id_utilisateur),

 CONSTRAINT fk_partage_animal
 FOREIGN KEY (id_animal)
 REFERENCES animal(id_animal)
 ON DELETE CASCADE,

 CONSTRAINT fk_partage_utilisateur
 FOREIGN KEY (id_utilisateur)
 REFERENCES utilisateur(id_utilisateur)
 ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE INDEX idx_partage_utilisateur
ON partage_animal(id_utilisateur);
```

Le MPD a été réalisé pour un SGBD relationnel de type MySQL/MariaDB (que je compte utilisé). Les clés primaires sont définies en AUTO_INCREMENT pour les tables principales, et en clé composite pour la table de partage. Les contraintes d'intégrité référentielle sont assurées via des clés étrangères avec suppression en cascade sur les relations liées à l'animal, afin d'éviter des données orphelines lors de la suppression d'un animal. Des index ont été ajoutés sur les colonnes utilisées fréquemment en jointures et filtres, notamment sur les événements par animal et par date, afin d'optimiser les requêtes liées à l'affichage du tableau de bord et aux rappels.

3.3 Justifications et vérification

Ce modèle répond directement aux besoins fonctionnels du projet. La relation propriétaire → animaux permet de gérer plusieurs animaux par utilisateur. La présence d'une table Evenement rattachée à un animal permet de conserver un historique complet, d'enregistrer des rendez-vous futurs et de piloter un système de rappels. L'entité Type_Evenement normalise les libellés et évite la duplication de catégories, ce qui améliore la cohérence des données. Enfin, la table Partage_Animal permet de gérer les accès multi-utilisateurs avec des droits différenciés, conformément au besoin de collaboration (famille, petsitter).
La structure respecte une logique de normalisation : chaque information est stockée une seule fois à l'endroit le plus pertinent, ce qui limite les redondances et facilite l'évolution future de l'application.